

Admin API — Frontend Integration Guide

This document is for frontend developers integrating the **admin service request decision screen** and **provider verification / provider profile** changes.

Base URL (local): `http://127.0.0.1:8010`

API prefix: `/api`

Auth: JWT — `Authorization: Bearer <access_token>`

Response envelope: Almost all admin endpoints return `{ "data": ... }`

Trailing slashes: Router uses `trailing_slash=False` — URLs do not end with `/`

1. Authentication

Login (admin)

```
POST /api/auth/login
```

Request body:

```
{
  "email": "admin@test.com",
  "password": "*****",
  "role": "admin"
}
```

Response (relevant fields):

```
{
  "access": "<jwt>",
  "refresh": "<jwt>",
  "role": "admin",
  "user": { "id": 2, "email": "admin@test.com", "full_name": "Admin User" }
}
```

Use `access` on every admin request.

2. Admin service requests — new & updated endpoints

All routes below require `IsAdmin` (`role === "admin"`).

2.1 List / standard detail (unchanged shape)

Method	URL	Body
GET	/api/admin-service-requests	—
GET	/api/admin-service-requests/{id}	—

List supports filters: `?status=`, `?category=`, `?created_at=`

Standard detail still uses `AdminServiceRequestSerializer` and includes the four signing booleans:

- `contract_signed_by_customer`
 - `contract_signed_by_provider`
 - `waiver_signed_by_customer`
 - `waiver_signed_by_provider`
-

2.2 Customer-side view (updated)

```
GET /api/admin-service-requests/{id}/customer-side
```

No request body.

Purpose: What the **customer** entered / sees for this request (not provider job photos).

Response shape:

```
{
  "data": {
    "entered": {
      "category": { "id": 1, "name": "Plumbing" },
      "city": "Islamabad",
      "exact_location": "33.6844,73.0479",
      "service_address": "...",
      "scheduled_at": null,
      "service_place_id": null,
      "service_place_name": null,
      "service_place_formatted_address": null,
      "service_place_latitude": null,
      "service_place_longitude": null,
      "payment_method": "card",
      "payment_status": "paid",
      "contract_signed_by_customer": true,
      "waiver_signed_by_customer": true,
      "contract": { "...see $2.5 contract (customer field"
      "waiver": { "...see $2.5 waiver (customer fields)..
      "needs_review": false
    },
    "phase_badge": {
      "step": 2,
      "total_steps": 5,
      "label": "Contract & waiver"
    }
  }
}
```

Changes from before:

- **Removed:** `photos` (photos are provider-uploaded only).
 - **Added:** nested `contract` and `waiver` detail objects (null if no row exists).
-

2.3 Provider-side view (updated)

```
GET /api/admin-service-requests/{id}/provider-side
```

No request body.

Purpose: What the **matched provider** entered / did on the job.

Response shape:

```
{
  "data": {
    "entered": {
      "matched_provider": { "id": 6, "business_name": "Ac
      "quoted_price": "1500.00",
      "contract_signed_by_provider": true,
      "waiver_signed_by_provider": true,
      "contract": { "...see $2.5 contract (provider field
      "waiver": { "...see $2.5 waiver (provider fields)..
      "en_route_at": "2026-07-15T11:00:00Z",
      "arrived_at": "2026-07-15T11:25:00Z",
      "started_at": "2026-07-15T11:28:00Z",
      "completed_at": "2026-07-15T13:45:00Z",
      "photos": [
        {
          "id": 1,
          "kind": "before",
          "image": "https://.../media/booking/photos/befo
          "created_at": "...
        }
      ]
    }
  }
}
```

```

    },
    "phase_badge": {
      "step": 5,
      "total_steps": 5,
      "label": "Completed"
    }
  }
}

```

Changes from before:

- **Added:** nested `contract` and `waiver` detail (provider-relevant fields).
- **Unchanged:** `photos` (before/after job images).

2.4 Full-detail — decision screen (NEW)

```
GET /api/admin-service-requests/{id}/full-detail
```

No request body.

Purpose: Single screen for admin to decide **refund** / **cancel** / **investigate**. Merges customer + provider data, timeline, auto-flags, and financials.

Response shape:

```

{
  "data": {
    "customer": { "...same as customer-side `entered`..."
    "provider": { "...same as provider-side `entered`..."
    "timeline": [
      {
        "event": "contract_signed_by_provider",
        "at": "2026-07-15T09:55:00Z",
        "source": "provider"
      },

```

```
{
  "event": "waiver_signed_by_provider",
  "at": "2026-07-15T09:57:00Z",
  "source": "provider"
},
{
  "event": "contract_signed_by_customer",
  "at": "2026-07-15T10:30:00Z",
  "source": "customer"
},
{
  "event": "waiver_signed_by_customer",
  "at": "2026-07-15T10:32:00Z",
  "source": "customer"
},
{
  "event": "en_route",
  "at": "2026-07-15T11:00:00Z",
  "source": "provider"
},
{
  "event": "arrived",
  "at": "2026-07-15T11:25:00Z",
  "source": "provider"
},
{
  "event": "started",
  "at": "2026-07-15T11:28:00Z",
  "source": "provider"
},
{
  "event": "completed",
  "at": "2026-07-15T13:45:00Z",
  "source": "provider"
},
{
  "event": "matched",
  "at": null,
  "source": "provider"
}
```

```

],
"flags": [
  {
    "type": "signature_mismatch",
    "message": "Contract signed flags differ between
  }
],
"financials": {
  "quoted_price": "1500.00",
  "payment_method": "card",
  "payment_status": "paid",
  "payment": {
    "status": "success",
    "amount": "1650.00",
    "paid_at": "2026-07-15T10:45:00Z"
  },
  "refundable_amount": "1650.00",
  "previous_refunds": null
},
"phase_badges": {
  "customer": { "step": 5, "total_steps": 5, "label":
  "provider": { "step": 5, "total_steps": 5, "label":
}
}
}

```

Timeline event values

event	source	at
matched	provider	always null (no matche
contract_signed_by_customer	customer	BookingContract.cus
contract_signed_by_provider	provider	BookingContract.pro
waiver_signed_by_customer	customer	BookingWaiver.custo
waiver_signed_by_provider	provider	BookingWaiver.provi
en_route	provider	ServiceRequest.en_r

arrived	provider	ServiceRequest.arrival
started	provider	ServiceRequest.start
completed	provider	ServiceRequest.complete
cancelled	system	ServiceRequest.cancel

Events with no timestamp are omitted except `matched` (included with `at: null` when a provider is matched).

Auto-flags (`flags []`)

type	When it appears
stalled_arrival	<code>arrived_at</code> set, <code>started_at</code> null, not completed/cancelled, >30 min elapsed
stalled_en_route	<code>en_route_at</code> set, <code>arrived_at</code> null, status still <code>en_route</code> , >2 hours elapsed
payment_mismatch	<code>payment_status</code> <code>=== "paid"</code> but no payment row or <code>payment.status</code> <code>!= "success"</code>
signature_mismatch	<code>contract_signed_by_customer</code> <code>!= contract_signed_by_provider</code>
cancelled_but_paid	<code>status</code> <code>=== "cancelled"</code> and <code>payment_status</code> <code>=== "paid"</code>

Empty array `[]` is valid (no issues detected).

Financials notes

- `payment.amount` and `refundable_amount` include the **10% platform fee** (total charged to card), not just `quoted_price`.
 - `previous_refunds` is always `null` today — no refund history table exists yet.
 - `payment` is `null` if no `ServiceRequestPayment` row.
-

2.5 Nested contract / waiver objects

At most **one** contract and **one** waiver per service request. If none exists, field is `null`.

Contract — customer nested fields (`customer.contract` or `customer-side`)

```
{
  "id": 7,
  "template": { "title": "Standard Contract", "version":
  "template_name": "Standard Contract",
  "template_version": 2,
  "content_snapshot": "...",
  "rendered_content": "<p>...</p>",
  "customer_agreed_at": "2026-07-15T10:30:00Z",
  "customer_sign_ip": "203.0.113.42",
  "customer_signature": {
    "type": "typed_name",
    "typed_name": "Ali Hassan",
    "image_url": null,
    "signed_at": "2026-07-15T10:30:00Z"
  },
  "status": "both_signed"
}
```

Contract status values: `pending` | `provider_signed` | `both_signed`

Contract — provider nested fields (`provider.contract` or `provider-side`)

Same as above but with `provider_agreed_at`, `provider_sign_ip`, `provider_signature` instead of customer fields. **status is the same row** — use it on either side to know if both signed.

Waiver — customer / provider nested fields

```
{
  "id": 4,
  "template": { "title": "Liability Waiver", "version": 1,
  "template_name": "Liability Waiver",
  "template_version": 1,
  "content_snapshot": "...",
  "rendered_content": "<p>...</p>",
  "customer_agreed_at": "...",
  "customer_sign_ip": "...",
  "customer_signature": { "type": "...", "typed_name": "...",
  "both_signed": true
}
```

Provider side mirrors with `provider_*` fields. `both_signed: true` means both parties signed the waiver.

How to show “both signed” in the UI

Document	Quick check (booleans on SR)	Authoritative check (nested row)
Contract	<code>contract_signed_by_customer</code> && <code>contract_signed_by_provider</code>	<code>contract.status</code> === <code>"both_signed"</code>
Waiver	<code>waiver_signed_by_customer</code> && <code>waiver_signed_by_provider</code>	<code>waiver.both_signed</code> === <code>true</code>

On **full-detail**, read booleans from `data.customer + data.provider`, or nested `contract/waiver` on either side.

2.6 Admin actions (unchanged — for context)

Method	URL	Body
--------	-----	------

POST	/api/admin-service-requests/{id}/cancel	{ "refund": true } optional (default true)
POST	/api/admin-service-requests/{id}/refund	{ "amount": "100.00" } optional (full refund if omitted)
POST	/api/admin-service-requests/{id}/reassign	{ "provider": <Provider.id> } required

Cancel/refund responses return { "data":
<AdminServiceRequestSerializer> }.

2.7 Phase badges

Customer and provider labels differ for some statuses (e.g. customer `open` → “Finding provider”, provider `open` → “Available to offer”).

- Customer-side: `data.phase_badge`
- Provider-side: `data.phase_badge`
- Full-detail: `data.phase_badges.customer` and `data.phase_badges.provider`

Each badge: { "step": 1-5, "total_steps": 5, "label": "..."} (step: 0 for cancelled).

3. Admin provider profile — license fields (NEW on detail)

When admin opens a **provider profile**, license data from verification is now included on the **detail** endpoint only (not list).

List providers

```
GET /api/admin-providers
```

Response fields (per item): id, business_name, user, rating, is_available, is_payout_enabled, profile_template, verification_status

verification_status: pending | approved | rejected | not_submitted

No license files on list.

Provider detail (includes licenses)

```
GET /api/admin-providers/{providerId}
```

No request body.

New fields on detail (from ProviderVerification):

Field	Type	Description
business_license	string null	Absolute URL to uploaded business license file
contractor_license_doc	string null	Absolute URL to contractor license document (optional upload)
contractor_license_number	string null	Contractor license number (text)

Example:

```
{
```

```

"data": {
  "id": 6,
  "business_name": "Acme Services",
  "user": {
    "id": 12,
    "email": "provider@example.com",
    "fullname": "Acme Owner",
    "is_active": true,
    "date_joined": "...",
  },
  "rating": "5.00",
  "is_available": true,
  "is_payout_enabled": false,
  "profile_template": "Default",
  "verification_status": "approved",
  "bio": "...",
  "years_of_experience": 5,
  "base_rate": "500.00",
  "language_spoken": ["English"],
  "within_radius": 10,
  "address": "...",
  "cnic_number": null,
  "business_license": "http://127.0.0.1:8010/media/prov
  "contractor_license_doc": "http://127.0.0.1:8010/medi
  "contractor_license_number": "LIC-12345",
  "stripe_account_id": null,
  "current_location": "33.6844,73.0479"
}
}

```

If the provider has **no** verification record, all three license fields are null.

UI suggestion: Show three rows on provider detail:

1. **Business license** — link/preview from `business_license`
 2. **Contractor license document** — link/preview from `contractor_license_doc` (hide or show “Not uploaded” if null)
 3. **Contractor license number** — display `contractor_license_number` as text
-

Other admin provider actions

Method	URL	Body
PATCH	/api/admin-providers/{id}	{ "is_available": true }
POST	/api/admin-providers/{id}/suspend	{}
POST	/api/admin-providers/{id}/reactivate	{}

Suspend/reactivate responses use the same **detail** serializer (includes license fields).

4. Provider verification — admin review flow

Separate from provider profile: full verification queue for approve/reject.

List verifications

```
GET /api/admin-verifications
```

Filters: ?status=pending|approved|rejected

Search: ?search= (business name, provider email)

Verification detail

```
GET /api/admin-verifications/{verificationId}
```

Response includes:

```
{
  "data": {
    "id": 3,
    "provider": {
      "id": 6,
      "business_name": "Acme Services",
      "email": "provider@example.com"
    },
    "business_license": "/media/provider/verification/bus
    "contractor_license_number": "LIC-12345",
    "contractor_license_doc": "/media/provider/verificati
    "applied_subcategories": [
      { "id": 2, "name": "Drain Cleaning", "parent": { "i
    ],
    "status": "pending",
    "review_note": "",
    "reviewed_by": null,
    "reviewed_at": null,
    "submitted_at": "..."}
  }
}
```

Note: On admin-verifications, file fields may be **relative** / media/... paths. On admin-providers/{id} detail they are **absolute URLs**. Prepend your API origin for relative paths if needed.

Approve / reject

```
PATCH /api/admin-verifications/{verificationId}
```

Request body:

```
{
  "status": "approved",
  "review_note": "Documents verified."
}
```

status must be "approved" or "rejected". review_note optional but recommended for rejections.

5. Provider-side verification (for context)

Providers submit/update their own verification (not admin):

Method	URL	Notes
GET	/api/provider-verification	Current provider's verification
POST	/api/provider-verification	Create (multipart)
PATCH	/api/provider-verification/{id}	Update docs/number/categories

Create body (multipart/form-data):

- business_license (file, required)
- contractor_license_number (string, required)
- contractor_license_doc (file, optional)
- applied_subcategory_ids (array of category IDs, required, min 1)

If a provider updates verification after **approved** or **rejected**, status resets to **pending** for re-review.

6. Recommended admin UI flows

Service request refund/cancel screen

1. GET /api/admin-service-requests/{id}/full-detail

2. Render `customer + provider` columns from `data.customer / data.provider`
3. Show `timeline` as chronological stepper
4. Highlight `flags` as warning chips
5. Show `financials.refundable_amount` before offering refund
6. Actions: POST `.../cancel` or POST `.../refund`

Provider admin profile

1. GET `/api/admin-providers` → table with `verification_status`
2. Row click → GET `/api/admin-providers/{id}`
3. Display **business license, contractor license doc, contractor license number**
4. Optional link → GET `/api/admin-verifications?search=` or open verification by provider

Verification queue

1. GET `/api/admin-verifications?status=pending`
 2. PATCH `/api/admin-verifications/{id}` to approve/reject
-

7. TypeScript-friendly interfaces (sketch)

```
type PhaseBadge = { step: number; total_steps: number; la

type SignaturePayload = {
  type: string | null;
  typed_name: string | null;
  image_url: string | null;
  signed_at: string | null;
};

type AdminContractDetail = {
```

```

id: number;
template: { title: string; version?: number } | null;
template_name: string | null;
template_version: number | null;
content_snapshot: string;
rendered_content: string | null;
status: "pending" | "provider_signed" | "both_signed";
customer_agreed_at?: string | null;
customer_sign_ip?: string | null;
customer_signature?: SignaturePayload;
provider_agreed_at?: string | null;
provider_sign_ip?: string | null;
provider_signature?: SignaturePayload;
};

type TimelineEvent = {
  event: string;
  at: string | null;
  source: "customer" | "provider" | "system";
};

type AdminFlag = { type: string; message: string };

type FullDetailResponse = {
  data: {
    customer: Record<string, unknown>;
    provider: Record<string, unknown>;
    timeline: TimelineEvent[];
    flags: AdminFlag[];
    financials: {
      quoted_price: string | null;
      payment_method: string | null;
      payment_status: string | null;
      payment: { status: string; amount: string; paid_at: string; refundable_amount: string | null; previous_refunds: null; };
    };
    phase_badges: { customer: PhaseBadge; provider: PhaseBadge };
  };
};

```

```
type AdminProviderDetail = {
  data: {
    id: number;
    business_name: string;
    verification_status: string;
    business_license: string | null;
    contractor_license_doc: string | null;
    contractor_license_number: string | null;
    // ...other provider fields
  };
};
```

8. Swagger

Interactive docs: `GET http://127.0.0.1:8010/` (Swagger UI, OpenAPI).

9. Backend source files (for reference)

Area	Files
Service request admin views	<code>api/booking/views.py</code> → <code>AdminServiceRequestViewSet</code>
Service request serializers	<code>api/booking/serializers.py</code> → <code>Admin*Side*</code> , <code>AdminFullDetailSerializer</code>
Timeline / flags / financials	<code>api/booking/</code> <code>service_request_customer.py</code>
Admin provider detail	<code>api/users/serializers.py</code> → <code>AdminProviderDetailSerializer</code>

Admin verification	<code>api/users/serializers.py</code> → <code>AdminVerificationSerializer</code>
Provider verification (provider)	<code>api/users/views.py</code> → <code>ProviderVerificationViewSet</code>
URL registration	<code>api/urls.py</code>

Last updated: admin service request full-detail + customer/provider side contract/waiver nesting + admin provider license fields on detail.